

实验报告

课程名称: 基于软硬件协同设计的 FPGA 项目实战

实验名称: 交通灯

实验日期: 2025 年 3 月 8 日 地 点: K421 实验台号:

专业班级: 2201110 学 号: 220111012 姓名: 王靳

评分:

教师评语:

教师签字:

日 期:

一、 实验目的（5 分）

掌握最基本的状态机的写法，重温 fpga 的开发。

二、 实验环境（5 分）

硬件平台：易灵思

软件工具：chisel, verilator, gtkwave, efinity, vscode

三、 实验原理（10 分）

（请**简要**概括本次实验涉及到的理论知识、基本概念、设计方法等。）

理论知识：上板子，调试对应的硬件（这个比较麻烦）。状态机的写法。

四、实验步骤

1、系统方案设计（20 分）

（用以说明模块划分及模块间接口的系统框图,说明各模块功能及输入输出信号,状态转换图、时序波形图等。）

traffic, 用于逻辑上的接受时钟/复位信号。

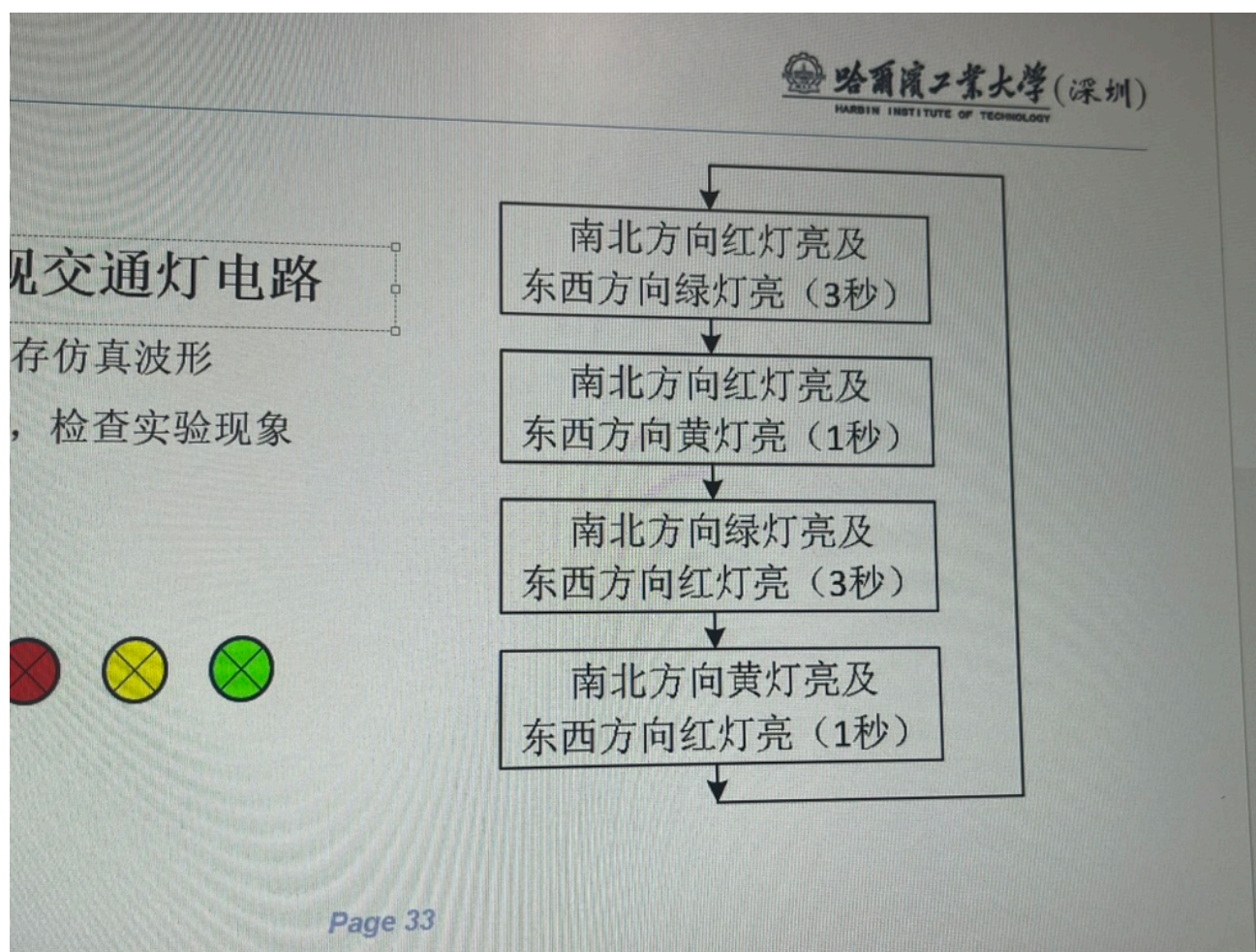
```
generated > TopModule.sv
1  // Generated by CIRCT firtool-1.62.1
2  module TrafficLight(
3      input  clock,
4      | | | reset,
5      output io_ns_red,
6      | | | io_ns_yellow,
7      | | | io_ns_green,
8      | | | io_we_red,
9      | | | io_we_yellow,
10     | | | io_we_green
11 );
12
```

TopModule, 实例化 trafficlight, 并绑定与开发板相关的引脚。

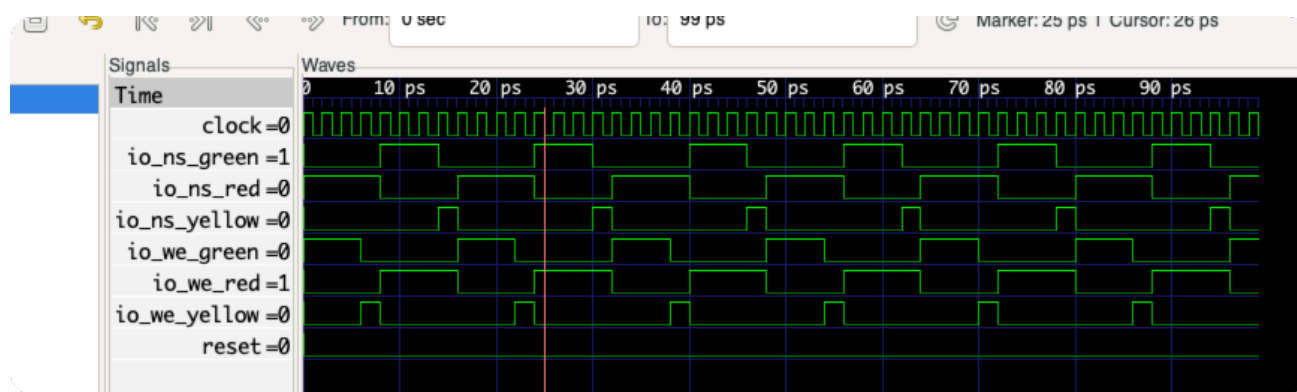
```
module TopModule(
    input          io_pll_inst1_CLKOUT0,
    | | | | | | | io_pll_inst1_LOCKED,
    output [7:0] io_led_data
);

    wire _led_io_ns_red;
    wire _led_io_ns_yellow;
    wire _led_io_ns_green;
    wire _led_io_we_red;
    wire _led_io_we_yellow;
    wire _led_io_we_green;
    TrafficLight led (
        .clock          (io_pll_inst1_CLKOUT0),
        .reset           (~io_pll_inst1_LOCKED),
        .io_ns_red       (_led_io_ns_red),
        .io_ns_yellow    (_led_io_ns_yellow),
        .io_ns_green     (_led_io_ns_green),
        .io_we_red        (_led_io_we_red),
        .io_we_yellow    (_led_io_we_yellow),
        .io_we_green     (_led_io_we_green)
    );
    assign io_led_data =
        {2'h0,
         ~_led_io_ns_red,
         ~_led_io_ns_yellow,
         ~_led_io_ns_green,
         ~_led_io_we_red,
         ~_led_io_we_yellow,
         ~_led_io_we_green};
endmodule
```

状态转移图基本上是这样。但是状态转移条件会随着中间的一个计数器而变化



时序波形图



2、代码实现（20 分）

（请以开发软件界面截图形式上传设计代码，关键代码**需要**有注释；说明输入输出端口的封装引脚、I/O Bank 及 I/O Voltage。）

TrafficLight

```

2
3 package led
4
5 import chisel3._
6 import chisel3.util._
7
8 object TrafficLightEnum extends ChiselEnum {
9   val ns_red_we_green, ns_red_we_yellow, ns_green_we_red, ns_yellow_we_red =
10     Value
11 }
12
13 class TrafficLight(delay: Int) extends Module {
14   val io = IO(new Bundle {
15     val ns_red = Output(Bool())
16     val ns_yellow = Output(Bool())
17     val ns_green = Output(Bool())
18     val we_red = Output(Bool())
19     val we_yellow = Output(Bool())
20     val we_green = Output(Bool())
21   })
22
23   val cnt = Counter(delay) // 100M * 1s
24
25   // 用于指示持续的时间
26   val last = Counter(3)
27   // %i to chat, %K to generate
28   val state = Reg(TrafficLightEnum())
29
30   // 状态切换
31   when(cnt.inc()) {
32     switch(state) {
33       is(TrafficLightEnum.ns_red_we_green) {
34         when(last.inc()) {
35           state := TrafficLightEnum.ns_red_we_yellow
36         }
37       }
38       is(TrafficLightEnum.ns_red_we_yellow) {
39         state := TrafficLightEnum.ns_green_we_red
40       }
41       is(TrafficLightEnum.ns_green_we_red) {
42         when(last.inc()) {
43           state := TrafficLightEnum.ns_yellow_we_red
44         }
45       }
46       is(TrafficLightEnum.ns_yellow_we_red) {
47         state := TrafficLightEnum.ns_red_we_green
48       }
49     }
50   }
51
52   // 状态输出
53   io.ns_red := state === TrafficLightEnum.ns_red_we_green || state === TrafficLightEnum.ns_red_we_yellow
54   io.ns_yellow := state === TrafficLightEnum.ns_yellow_we_red
55   io.ns_green := state === TrafficLightEnum.ns_green_we_red
56   io.we_red := state === TrafficLightEnum.ns_green_we_red || state === TrafficLightEnum.ns_yellow_we_red
57   io.we_yellow := state === TrafficLightEnum.ns_red_we_yellow
58   io.we_green := state === TrafficLightEnum.ns_red_we_green
59
60 }

```

TopModule，用于绑定时钟等，以及适应低电平触发等硬件特性。

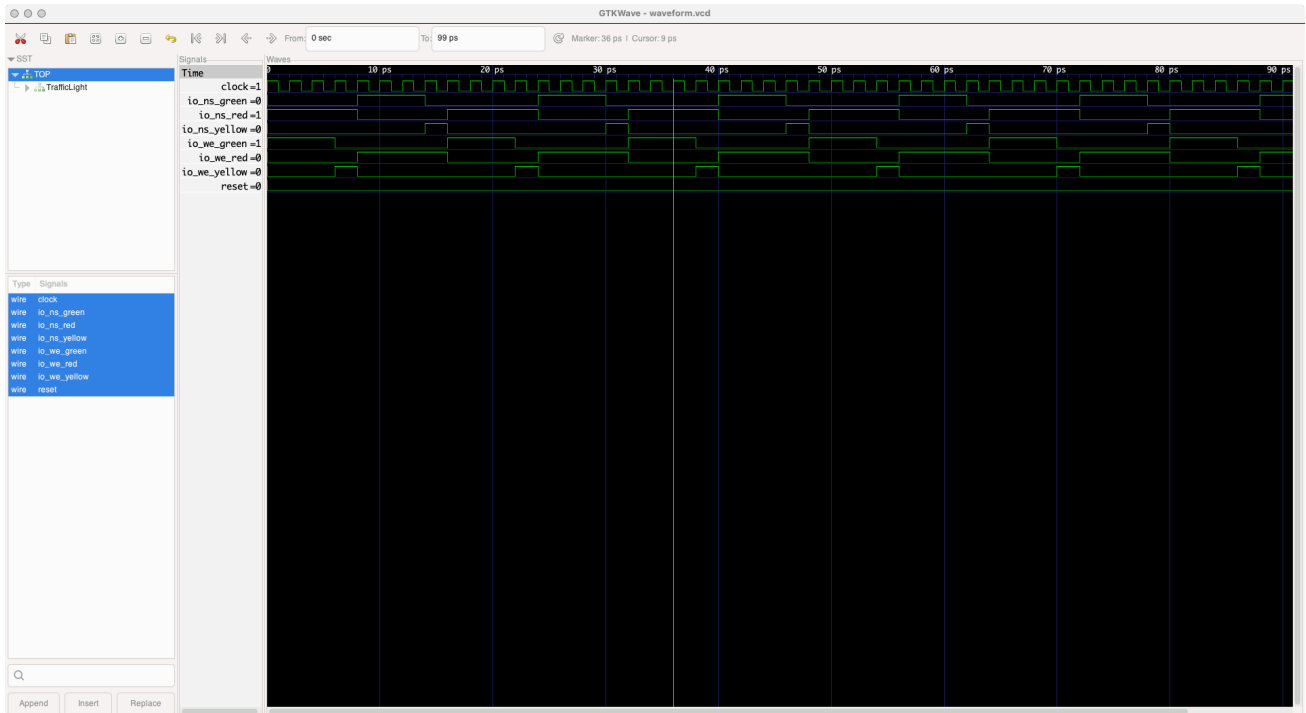
```
4
5 class TopModule extends RawModule {
6   val io = IO(new Bundle {
7     val pll_inst1_CLKOUT0 = Input(Clock())
8     val pll_inst1_LOCKED = Input(Bool())
9     val led_data = Output(UInt(8.W))
10  })
11
12  val led =
13    withClockAndReset(io.pll_inst1_CLKOUT0, !io.pll_inst1_LOCKED) {
14      Module(new TrafficLight(100_000_000))
15    }
16
17  io.led_data := false.B ## false.B ## !led.io.ns_red ## !led.io.ns_yellow ## !led.io.ns_green ## !led.io.we_red ## !led.io.we_yellow ## !led.io.we_green
18
19 }
20
```

3、测试文件及仿真结果（20 分）

（请以开发软件界面截图形式上传仿真代码，关键代码需要有注释；上传并分析仿真波形图，说明仿真结果。）


```
src > test > cxx >  main.cxx > ...
1  #include "VTrafficLight.h"
2  #include "verilatedos.h"
3  #include <verilated.h>
4  #include <verilated_vcd_c.h>
5
6  vluint64_t sim_time = 0;
7
8  int main(int argc, char **argv, char **env) {
9      VTrafficLight *dut = new VTrafficLight;
10
11      Verilated::traceEverOn(flag: true);
12      VerilatedVcdC *m_trace = new VerilatedVcdC;
13      dut->trace(tfp: m_trace, levels: 5);
14      m_trace->open(filename: "waveform.vcd");
15
16      for (int i = 0; i < 100; i++) {
17          dut->clock ^= 1;
18          dut->eval();
19          m_trace->dump(timeui: sim_time);
20          sim_time++;
21      }
22      m_trace->close();
23      delete dut;
24      exit(status: EXIT_SUCCESS);
25  }
26   to chat,  to generate
```

波形图



0' state=南北红，东西绿；io_ns_red=1, io_we_green=1 符合预期

6' 经过 3 个周期，状态转移，state=南北红，东西黄；io_ns_red=1, io_we_green=0, io_we_yellow=1, 符合预期

8' 经过 1 个周期，状态转移，state=南北绿，东西红；io_ns_green=1, io_we_red=1, 符合预期

14' 经过 3 个周期，状态转移，state=南北宏，东西红；io_ns_yellow=1, io_we_red=1, 符合预期

五、实验结果（10 分）

（描述实验现象，说明与预期是否一致。）

现象，6 个 led，能以 3-1-3-1 的频率，两两亮起，对应的也就是 南北红黄绿，东西红黄绿。
现象与预期一致

六、实验总结（10 分）

（记录实验过程中遇到的问题及解决方法，总结设计经验。）

教训：不应该在 macOS 上配置 efinity 环境